INSA Strasbourg • FIP GE4 • Informatique 2 — Langage C/C++

Projet — CellSight Lite — explorer les données de cyclage du laboratoire

Objectif

En binôme, vous allez terminer **CellSight Lite**, une application de bureau C++/Qt qui permet au laboratoire d'explorer les données de cyclage de ses cellules lithium-ion : fiche de chaque cellule, courbes de mesures, indicateurs de vieillissement (capacité, SOH, résistance interne, état de charge), vérification des plans de test et exports CSV/MATLAB.
L'architecture est fournie : toutes les classes sont déclarées, l'interface graphique est presque complète et une batterie de tests vérifie votre travail étape par étape. Vous écrivez le cœur du logiciel — et vous mettez en œuvre tout le programme du cours : classes, héritage, polymorphisme, exceptions, STL, fichiers et Qt.

Table des matières

1 Contexte : des années de mesures que personne n'exploite **2**

2 Organisation **2**
2.1 Travail en binôme . . . 2
2.2 Les étapes . . . 3
2.3 Livrables et évaluation . . . 3

3 Le minimum à savoir sur les batteries **4**

4 Les données du laboratoire **5**

5 L'architecture fournie **6**
5.1 Les classes . . . 6
5.2 Le dossier du projet . . . 6
5.3 Le mécanisme des « TODO » . . . 6

6 Les étapes **8**
0 Mise en route . . . 8
1 Classes métier : Datasheet, Cell, User . . . 9
2 Séries temporelles avec la STL . . . 9
3 Importer les fichiers BaSyTec . . . 10
4 Bancs de cyclage et polymorphisme . . . 11
5 Les indicateurs de vieillissement . . . 12
6 État de charge et sécurité des plans de test . . . 13
7 Export CSV . . . 14
8 L'interface graphique . . . 14
9 Bonus . . . 15

7 Et après ? Votre code dans CellSight **16**

A Problèmes fréquents **16**

1. Contexte : des années de mesures que personne n'exploite

Le laboratoire fait vieillir des cellules lithium-ion sur des *bancs de cyclage* (des « cycleurs ») de deux constructeurs, **Arbin** et **BaSyTec**. Chaque cellule est chargée puis déchargée des centaines de fois pendant des mois ; le cycleur enregistre courant, tension et température toutes les minutes. Les fichiers s'accumulent sur des disques durs et, faute d'outil, chaque courbe demande d'écrire un script… qu'on réécrit la fois suivante.

Le laboratoire développe donc **CellSight**, une plateforme qui centralise ces données et les exploite (figure 1). Vous en construisez la partie C++ : **CellSight Lite**, l'application de bureau. Ce n'est pas un exercice jetable — les classes que vous écrivez (`Cell`, `Datasheet`, `TestBench`, `Metric`…) sont celles du modèle métier de la plateforme complète.

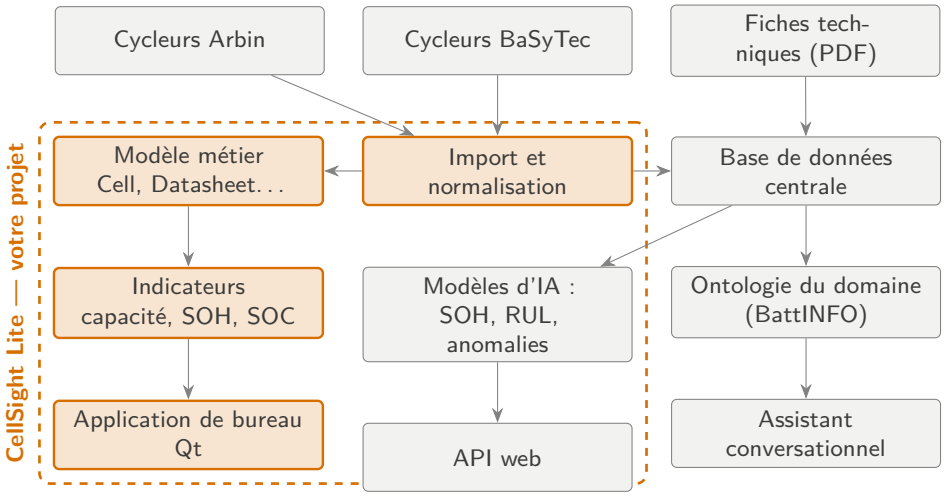


FIGURE 1 – La plateforme CellSight. En orange, ce que couvre CellSight Lite, votre projet C++. Le reste (base centrale, IA, ontologie, assistant) est développé en Python par le laboratoire et réutilise le même modèle métier.

Des données synthétiques. Les vraies données du laboratoire ne sortent pas du laboratoire. Vous travaillez sur un jeu **synthétique** : quatre cellules de fabricants fictifs, simulées par un modèle électrique équivalent avec vieillissement, au format des deux cycleurs. Avantage décisif : le générateur connaît la *vérité terrain* (la vraie capacité de chaque cycle, la vraie résistance…), ce qui permet aux tests de vérifier vos calculs au pour-mille près.

2. Organisation

2.1. Travail en binôme

Le projet se fait **en binôme**, et uniquement en binôme. Les deux membres écrivent du code et **les deux doivent pouvoir expliquer l'ensemble du projet** : lors de la soutenance, chacun sera interrogé sur une partie qu'il n'a pas forcément écrite. Une organisation qui marche bien :

- écrire ensemble les étapes 1 et 2 (elles fixent les bases) ;
- se répartir ensuite les étapes indépendantes (par exemple 3–4 d'un côté, 5–6 de l'autre), puis **relire le code de l'autre** avant de valider l'étape ;
- faire l'étape 8 (interface) ensemble.

Utilisez **Git** (un dépôt par binôme, sur la forge de l'école ou sur GitHub en privé) : chaque étape terminée fait l'objet d'au moins un commit, et l'historique doit montrer les contributions des deux membres.

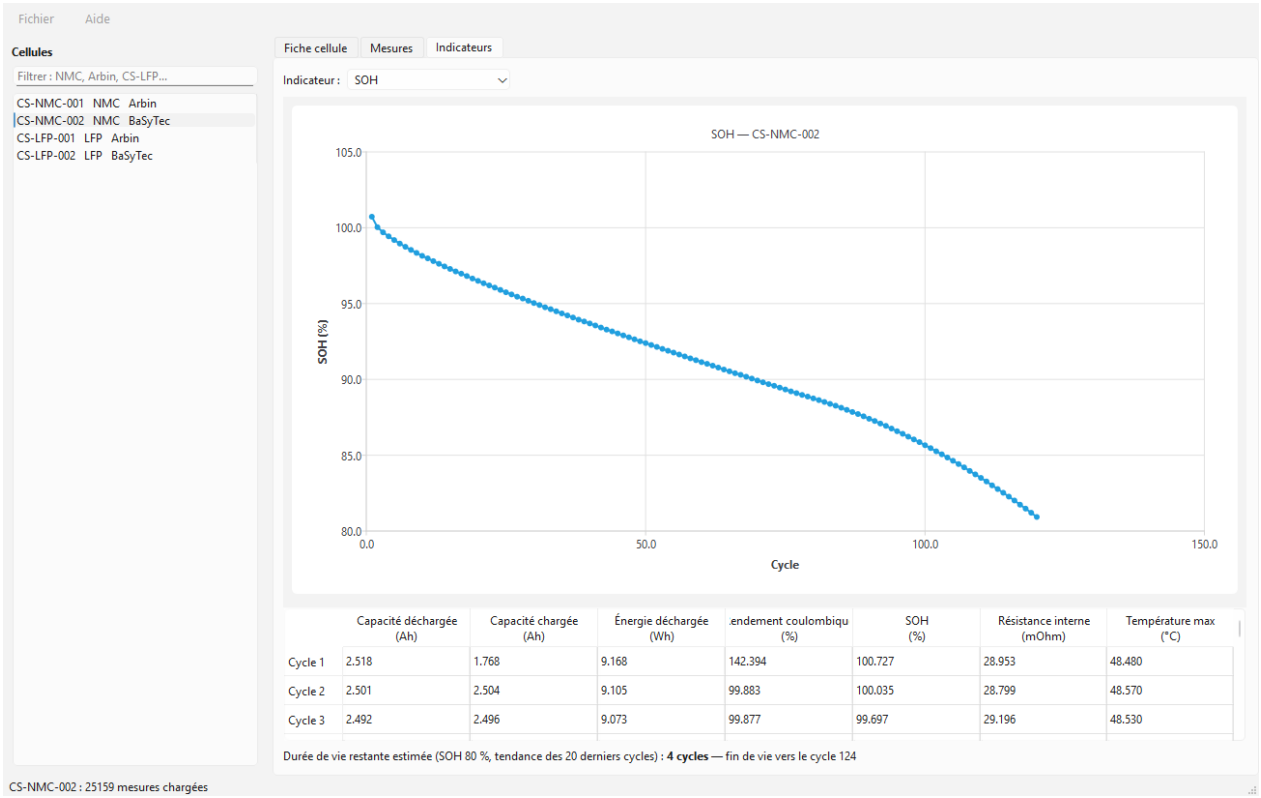


FIGURE 2 – L'application terminée : SOH d'une cellule vieillie à 45 °C, tableau des indicateurs par cycle et estimation de la durée de vie restante.

2.2. Les étapes

Chaque étape correspond à des notions du cours et à un groupe de tests. Les durées sont indicatives.

Étape	Thème	Notions du cours	Durée
0	Mise en route	Qt Creator, CMake, tests	1 h
1	Classes métier	classes, constructeurs, encapsulation, exceptions (cours 2–3)	2 h
2	Séries temporelles	STL : `vector`, `set`, algorithmes, lambdas (cours 3)	2 h
3	Import BaSyTec	héritage, fichiers, analyse de texte (cours 2–3)	3 h
4	Bancs de cyclage	polymorphisme, méthode de fabrique, `unique_ptr` (cours 2)	2 h
5	Indicateurs	classes abstraites, patron Stratégie, calcul numérique	4 h
6	SOC et sécurité	algorithme sur une série, règles métier	3 h
7	Export CSV	flux de fichiers, interfaces	1 h
8	Interface graphique	signaux et slots, dialogues Qt (cours 4)	3 h
9	Bonus	régression, Qt, multitâche (cours 5)	—

2.3. Livrables et évaluation

Attention

Date de rendu : jeudi 7 janvier 2027 à 23 h 59 au plus tard. Le code et le rapport sont envoyés à l'enseignant (slimane.arbaoui@insa-strasbourg.fr) avant cette date. Les soutenances commencent le **vendredi 8 janvier 2027** : ce qui n'a pas été rendu à temps ne peut pas être présenté.

- **Le code** : le dépôt Git (ou une archive `.zip` du dossier, *sans* le dossier de compilation).

- **Un rapport court** (4 pages au plus) : qui a fait quoi, les choix faits, les difficultés rencontrées, des captures d'écran, et les bonus réalisés.
- **Une démonstration** de 10 minutes suivie de questions individuelles.

Critère	Points
Tests automatiques des étapes 1 à 7 (`cellsight_tests`)	10
Interface graphique (étape 8) : fonctionnelle et robuste	3
Qualité du code : lisibilité, `const`, aucun avertissement, historique Git	3
Démonstration et réponses aux questions individuelles	4
Total	**20**
Bonus (étape 9)	jusqu'à +2

Attention

Règles du jeu. Ne modifiez *ni les tests, ni les en-têtes* (`core/include`) : le correcteur utilisera les siens. Les données de `data/truth/` servent aux tests : votre code ne doit *jamais* les lire. Un projet qui ne compile pas n'est pas évalué.

3. Le minimum à savoir sur les batteries

Cellule, chimie, capacité. Une cellule lithium-ion est caractérisée par sa *chimie* (ici NMC et LFP), sa *capacité nominale* Q_{nom} en ampères-heures (Ah) — la charge qu'elle peut délivrer —, et sa fenêtre de tension $[V_{\min}, V_{\max}]$ à ne jamais quitter. Ces caractéristiques figurent dans la *fiche technique* (datasheet) du fabricant.

Régime C. Les courants s'expriment en multiples de la capacité : pour une cellule de 2,5 Ah, « 1C » vaut 2,5 A et « C/2 » vaut 1,25 A.

Un cycle. Le plan de test répète la même séquence (figure 3) : charge à courant constant (CC) jusqu'à $V_{\max}$, puis à tension constante (CV) jusqu'à ce que le courant tombe sous C/20 ; repos ; décharge CC jusqu'à $V_{\min}$; repos. Par convention, le courant est **positif en charge** et **négatif en décharge**.

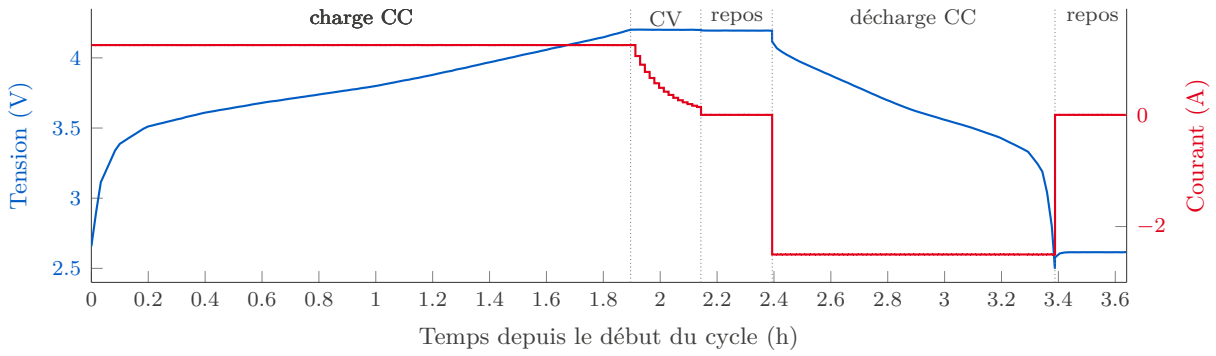


FIGURE 3 – Cycle 10 de la cellule CS-NMC-001 (données du projet) : tension en bleu, courant en rouge.

Les indicateurs que vous calculerez. Pour chaque cycle, avec t en secondes, I en ampères et V en volts :

Capacité déchargée	$Q_{\mathrm{d}} = \dfrac{1}{3600}\displaystyle\int \max(0,\,-I)\,\mathrm{d}t$ (Ah)
Capacité chargée	$Q_{\mathrm{c}} = \dfrac{1}{3600}\displaystyle\int \max(0,\,I)\,\mathrm{d}t$ (Ah)
Énergie déchargée	$E_{\mathrm{d}} = \dfrac{1}{3600}\displaystyle\int V\cdot\max(0,\,-I)\,\mathrm{d}t$ (Wh)
Rendement coulombique	$\eta = 100\,Q_{\mathrm{d}}/Q_{\mathrm{c}}$ (%)
État de santé (SOH)	$\mathrm{SOH} = 100\,Q_{\mathrm{d}}/Q_{\mathrm{nom}}$ (%)
Résistance interne	$R = \Delta V/\Delta I$ au passage repos → décharge (mΩ)

Une cellule vieillit : sa capacité baisse et sa résistance augmente. On la considère en fin de vie quand son SOH passe sous **80 %** ; le nombre de cycles qui lui restent avant ce seuil est sa *durée de vie restante* (*RUL*). L'*état de charge* (*SOC*, en %) dit à tout instant quelle fraction de la capacité est disponible, comme une jauge de carburant.

Indice

Deux curiosités des données, qui ne sont pas des erreurs. Au premier cycle, la cellule part d'un état de charge de 30 % : elle charge moins qu'elle ne décharge, et son rendement coulombique dépasse 100 %. Et une cellule neuve délivre souvent un peu plus que sa capacité nominale : un SOH de 101 % au début est normal.

4. Les données du laboratoire

Le dossier `data/` reproduit l'organisation du laboratoire :

```
data/
  cells.csv            une ligne par cellule : fiche, banc, voie, plan, fichier de mesures
  datasheets.csv       caractéristiques constructeur (le PDF est dans datasheets/)
  benches.csv          bancs de cyclage
  users.csv            utilisateurs et rôles
  plans/*.csv          plans de test (une étape par ligne)
  raw/arbin/*.csv      mesures brutes, format Arbin
  raw/basytec/*.txt    mesures brutes, format BaSyTec
  truth/*.csv          vérité terrain par cycle (réservée aux tests)
```

Cellule	Chimie	Banc	Ambiance	Cycles	Particularité
CS-NMC-001	NMC 2,5 Ah	Arbin	25 °C	120	vieillissement modéré
CS-NMC-002	NMC 2,5 Ah	BaSyTec	45 °C	120	vieillissement accéléré, proche de la fin de vie
CS-LFP-001	LFP 3,0 Ah	Arbin	25 °C	100	très stable
CS-LFP-002	LFP 3,0 Ah	BaSyTec	25 °C	100	décharge rapide à 2C

Deux cycleurs, deux formats. C'est tout le problème que résout l'étape d'import : les mêmes grandeurs, écrites différemment.

```
Data_Point,Test_Time(s),Date_Time,Step_Time(s),Step_Index,Cycle_Index,Current(A),Voltage(V),Aux_Temperature_1(C)
1,1.000,2026-01-12 09:00:01,1.000,1,1,1.2494,3.6576,24.96
2,61.000,2026-01-12 09:01:01,61.000,1,1,1.2504,3.6761,25.13
```

```
~Resultfile from CTS (synthetic CellSight export)
~Date of Test: 03.02.2026
~Time of Test: 08:30:00
~Battery: CS-NMC-002
~Channel: 5
~
~Time[h];DataSet;Command;U[V];I[A];T1[degC];Cyc-Count;Line
```

```
0.0002778;1;Charge;3.6564;1.2494;44.96;1;1
0.0169444;2;Charge;3.6733;1.2491;45.03;1;1
```

	Arbin (`.csv`)	BaSyTec (`.txt`)
Séparateur	virgule	point-virgule
En-tête	la première ligne	la **dernière** des lignes commençant par `~`
Temps	secondes	**heures**
Ordre des colonnes	le sien	un autre
Signe du courant	> 0 en charge	> 0 en charge

5. L'architecture fournie

5.1. Les classes

La figure 4 montre les classes du projet. Les classes grises sont **fournies** et complètes ; les classes orange contiennent des méthodes **à écrire**, marquées ⋆. Toutes les déclarations (fichiers `.hpp`) sont fournies : vous ne touchez qu'aux fichiers `.cpp`.

Quatre idées de conception à repérer — on vous interrogera dessus en soutenance :

- **Partage plutôt que copie.** Plusieurs cellules du même modèle partagent *la même* fiche technique : `Cell` la référence par `std::shared_ptr<const Datasheet>`.
- **Méthode de fabrique.** Le banc sait dans quel format il exporte : c'est lui qui fabrique l'importeur adapté (`createImporter`). Une cellule charge ses mesures sans savoir de quel cycleur elles viennent.
- **Patron Stratégie.** Chaque indicateur est une classe dérivée de `Metric`. Ajouter un indicateur, c'est écrire une classe et l'enregistrer : l'interface le propose alors toute seule.
- **Interface de dépôt.** L'interface graphique ne lit jamais un fichier : elle demande les cellules à un `CellRepository`. Dans CellSight, un autre dépôt interrogera le serveur central — sans changer une ligne de l'interface.

5.2. Le dossier du projet

```
CellSightLite/
  CMakeLists.txt
  core/include/cellsight/   les déclarations : NE PAS MODIFIER
  core/src/                 les implémentations : vos « TODO Étape N »
  tests/                    un fichier de tests par étape : NE PAS MODIFIER
  gui/                      l'interface Qt (étape 8)
  data/                     les données synthétiques
```

5.3. Le mécanisme des « TODO »

Chaque méthode à écrire contient, dans le kit, un commentaire et une exception :

```cpp
double DischargeCapacity::compute(const TimeSeries& cycle, const Datasheet& ds) const {
    // TODO Étape 5 — DischargeCapacity::compute : à écrire (voir le sujet).
    throw NotImplemented(5, "DischargeCapacity::compute");
}
```

L'exception `cs::NotImplemented` est interceptée partout : l'application se lance dès le premier jour et affiche « Étape 5 à implémenter » à la place de ce qui manque (figure 5). Chaque étape terminée fait apparaître de nouvelles fonctionnalités. Remplacez *tout* le corps de la méthode, `throw` compris, par votre code.

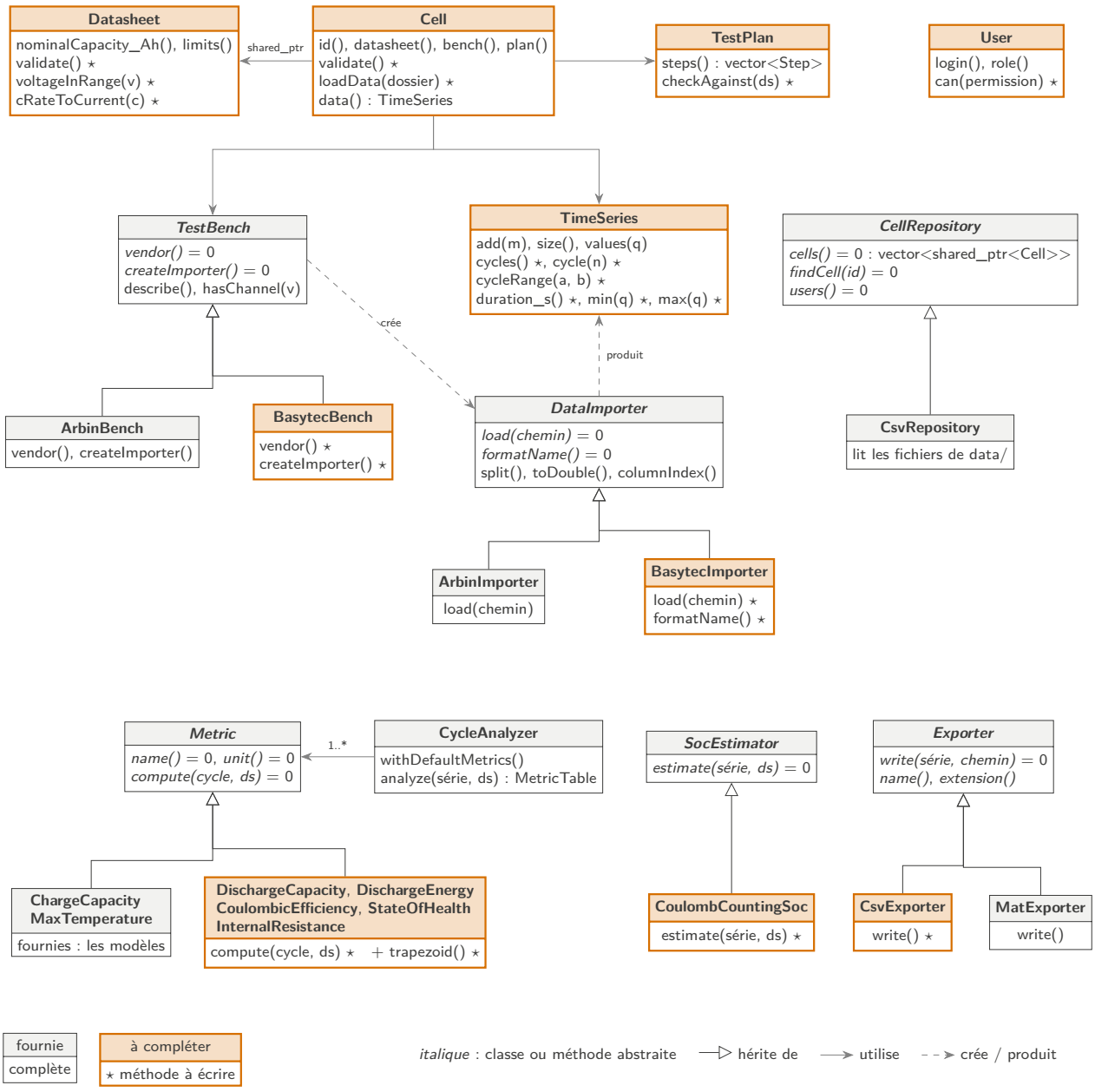


FIGURE 4 – Diagramme de classes de CellSight Lite (espace de noms `cs`). La fenêtre Qt (`MainWindow`) ne connaît que `CellRepository`, `Cell`, `CycleAnalyzer` et les `Exporter`.

6. Les étapes

Étape 0 — Mise en route

1. Installez **Qt 6** avec le *Qt Online Installer* (`https://www.qt.io/download-qt-installer`, compte gratuit). Dans la version LTS la plus récente, cochez **MinGW 64-bit** et **Qt Charts** ; dans *Developer and Designer Tools*, cochez **MinGW**, **CMake** et **Ninja**. Sous Linux, les paquets `qt6-base-dev` et `libqt6charts6-dev` suffisent ; sous macOS, le même installeur.
2. Ouvrez le projet : *Fichier > Ouvrir un fichier ou projet*, choisissez `CellSightLite/CMakeLists.txt`, acceptez le kit proposé, puis compilez (Ctrl+B).
3. Lancez la cible **`cellsight_tests`** (sélecteur de cible en bas à gauche). Elle affiche le bilan de toutes les étapes.
4. Lancez la cible **`cellsight_gui`**, connectez-vous et parcourez l'application.
5. Créez le dépôt Git du binôme et faites un premier commit.

Sortie attendue

```
[A FAIRE] etape 3  nom_du_format
           -> Etape 3 a implementer : BasytecImporter::formatName
[ECHEC  ] etape 1  datasheet_refuse_une_capacite_nulle_ou_negative
           -> test_etape1.cpp:49 : makeDatasheet(0.0) aurait du lever std::invalid_argument
...
Bilan                              reussis
Etape 1  Classes metier             2 / 12  ECHEC
Etape 2  Series temporelles         0 / 8   a faire
...
```

Trois verdicts : `OK` ; `A FAIRE` (le code lève encore `NotImplemented`) ; `ECHEC` (le code s'exécute mais donne un mauvais résultat — le message dit lequel et à quelle ligne du test). Pour ne lancer qu'une étape : argument `--etape 3` (dans Qt Creator : *Projets > Exécuter > Arguments*), ou `ctest -R etape3` en ligne de commande.

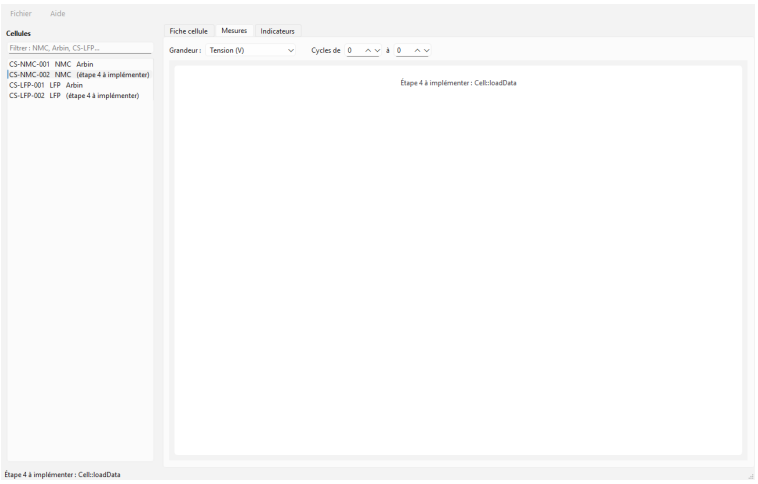


FIGURE 5 – Le kit au premier lancement : la liste des cellules et leur fiche s'affichent, mais les mesures attendent l'étape 4.

Étape 1 — Classes métier : Datasheet, Cell, User

Objectif

Garantir qu'un objet ne puisse *jamais* exister dans un état incohérent : c'est le rôle du constructeur, et c'est ce que permet l'encapsulation.

Fichiers : `core/src/Datasheet.cpp`, `Cell.cpp`, `User.cpp`.

1. `Datasheet::validate()`, appelée par le constructeur : lever `std::invalid_argument` si l'identifiant est vide, si la capacité nominale est ≤ 0, si $V_{\min} \geq V_{\max}$, ou si l'un des deux courants maximaux est ≤ 0. Le message doit dire *ce* qui ne va pas.
2. `Datasheet::voltageInRange(v)` : vrai si $V_{\min} \leq v \leq V_{\max}$ (bornes incluses).
3. `Datasheet::cRateToCurrent(c)` : le courant en ampères d'un régime c (1C = capacité nominale).
4. `Cell::validate()` : lever `std::invalid_argument` si l'identifiant est vide, si la fiche, le banc ou le plan est absent (pointeur nul), ou si la voie n'existe pas sur le banc (`TestBench::hasChannel`).
5. `User::can(p)` selon le tableau ci-dessous.

	`ViewData`	`ExportData`	`EditCatalog`
`Viewer` (visiteur)	oui	non	non
`Researcher` (chercheur)	oui	oui	non
`Admin` (responsable)	oui	oui	oui

Indice

Un `switch` sur un `enum class` est idéal pour `can` : avec `-Wall`, le compilateur signale un cas oublié. Pour un message d'erreur, `"Datasheet " + id_ + " : capacite nominale <= 0"` se construit directement avec des `std::string`.

Validation

`ctest -R etape1` — 12 tests. Dans l'application, l'onglet *Fiche* n'en dépend pas encore, mais l'utilisateur « invite » ne peut plus exporter.

Étape 2 — Séries temporelles avec la STL

Objectif

Manipuler une série de mesures avec les conteneurs et les algorithmes de la STL, sans réécrire de boucle quand un algorithme existe.

Fichier : `core/src/TimeSeries.cpp`. Une `TimeSeries` encapsule un `std::vector<Measurement>` (`data_`).

1. `cycles()` : les numéros de cycle présents, *distincts et triés*, même si les mesures arrivent dans le désordre.

2. `cycleRange(first, last)` : une nouvelle série ne contenant que les cycles `first` à `last` inclus (vide si `first > last`).
3. `cycle(n)` : les mesures du seul cycle `n` — réutilisez `cycleRange` !
4. `duration_s()` : temps de la dernière mesure moins temps de la première, 0 pour une série vide.
5. `min(q)` et `max(q)` : extrema d'une grandeur (`Quantity::Voltage`...). Sur une série vide : lever `std::logic_error`.

Indice

`std::set<int>` trie et dédoublonne en même temps. `std::copy_if` avec `std::back_inserter(out.data_)` et une lambda filtre une série. `std::max_element` accepte une lambda de comparaison ; la fonction libre `valueOf(m, q)` (fournie) lit la grandeur `q` d'une mesure. `TimeSeries::cycleRange` est une méthode de `TimeSeries` : elle a accès au `data_` privé de *l'autre* série qu'elle construit.

Validation

`ctest -R etape2` — 8 tests, dont un sur les 120 cycles de CS-NMC-001.

Étape 3 — Importer les fichiers BaSyTec

Objectif

Écrire une classe dérivée sur le modèle d'une classe sœur fournie, lire un fichier texte et convertir ses données en unités SI.

Fichier : `core/src/DataImporter.cpp`. L'importeur Arbin est fourni ; il est votre modèle :

```cpp
TimeSeries ArbinImporter::load(const std::string& path) const {
    std::ifstream in(path);
    // Sur MinGW, if (!in) ne détecte pas un fichier absent : tester is_open().
    if (!in.is_open()) {
        throw std::runtime_error("Impossible d'ouvrir " + path);
    }

    std::string line;
    if (!std::getline(in, line)) {
        throw std::runtime_error("Fichier vide : " + path);
    }
    // Les colonnes sont repérées par leur NOM : l'ordre peut changer.
    const std::vector<std::string> header = split(line, ',');
    const std::size_t iTime = columnIndex(header, "Test_Time(s)");
    const std::size_t iStep = columnIndex(header, "Step_Index");
    const std::size_t iCycle = columnIndex(header, "Cycle_Index");
    const std::size_t iCurrent = columnIndex(header, "Current(A)");
    const std::size_t iVoltage = columnIndex(header, "Voltage(V)");
    const std::size_t iTemp = columnIndex(header, "Aux_Temperature_1(C)");

    TimeSeries series;
    while (std::getline(in, line)) {
        if (line.empty()) {
            continue;
        }
        const std::vector<std::string> f = split(line, ',');
        if (f.size() != header.size()) {
```

```
            throw std::runtime_error("Ligne mal formee : " + line);
        }
        Measurement m;
        m.time_s = toDouble(f[iTime]);
        m.step = toInt(f[iStep]);
        m.cycle = toInt(f[iCycle]);
        m.current_A = toDouble(f[iCurrent]);  // Arbin : > 0 en charge, comme nous
        m.voltage_V = toDouble(f[iVoltage]);
        m.temperature_C = toDouble(f[iTemp]);
        series.add(m);
    }
    return series;
}
```

Écrivez `BasytecImporter::formatName()` (par exemple `"BaSyTec TXT"`) et `BasytecImporter::load(path)`, qui doit :

1. lever `std::runtime_error` si le fichier ne s'ouvre pas ;
2. lire les lignes qui commencent par `~` : *la dernière* d'entre elles, sans son `~`, donne les noms de colonnes ;
3. repérer les colonnes `Time[h]`, `U[V]`, `I[A]`, `T1[degC]`, `Cyc-Count` et `Line` *par leur nom* (`columnIndex`) — l'ordre peut changer ;
4. pour chaque ligne de données (séparateur `;`), remplir une `Measurement` : **le temps est en heures**, à convertir en secondes ; `Cyc-Count` est le cycle, `Line` l'étape.

Attention

Sous MinGW, `if (!in)` ne détecte *pas* un fichier absent : testez `in.is_open()` (voir cours 3). Et ne supposez pas l'ordre des colonnes : un test réordonne les colonnes et utilise des fins de ligne Windows.

Validation

`ctest -R etape3` — 7 tests. Le premier point de CS-NMC-002 doit valoir 1 s (et non 0,000278), +1,25 A et environ 45 °C.

Étape 4 — Bancs de cyclage et polymorphisme

Objectif

Exploiter le polymorphisme : un même appel, `bench().createImporter()`, fabrique l'importeur qui convient au banc réel de la cellule.

Fichiers : `core/src/TestBench.cpp`, `Cell.cpp`. Le banc Arbin est fourni :

```
std::string ArbinBench::vendor() const {
    return "Arbin";
}

std::unique_ptr<DataImporter> ArbinBench::createImporter() const {
    return std::make_unique<ArbinImporter>();
}
```

1. Écrivez `BasytecBench::vendor()` (`"BaSyTec"`, exactement) et `BasytecBench::createImporter()`.

2. Écrivez `Cell::loadData(dossier)` : demander un importeur au banc de la cellule, charger le fichier `dossier + "/" + dataFile()`, garder la série dans `data_` et noter qu'elle est chargée.

Indice

`Cell` ne contient *aucun* `if` sur le constructeur du banc : c'est tout l'intérêt. Si vous écrivez `if (bench_->vendor() == "Arbin")`, vous avez manqué l'idée.

Validation

`ctest -R etape4` — le dernier test charge les quatre cellules dans une seule boucle. Dans l'application, les courbes de tension apparaissent.

Étape 5 — Les indicateurs de vieillissement

Objectif

Implémenter des stratégies interchangeables derrière une classe abstraite, et calculer des grandeurs physiques justes à 0,5 % près.

Fichiers : `core/src/Integration.cpp`, `Metrics.cpp`.

1. `trapezoid(x, y)` : intégrale par la méthode des trapèzes, $\sum_k \frac{1}{2}(y_{k-1}+y_k)(x_k-x_{k-1})$. 0 avec moins de deux points ; `std::invalid_argument` si les tailles diffèrent.
2. `charge_Ah(série, sens)` : construire les vecteurs du temps et de $\max(0, \pm I)$ selon le sens, intégrer, diviser par 3600.
3. `energy_Wh(série, sens)` : idem avec $V \cdot \max(0, \pm I)$.
4. Les cinq indicateurs : `DischargeCapacity`, `DischargeEnergy`, `CoulombicEfficiency` (NaN si rien n'a été chargé), `StateOfHealth` et `InternalResistance`.

Les deux indicateurs fournis montrent la forme attendue, et la fabrique `withDefaultMetrics()` les enregistre tous :

```cpp
double ChargeCapacity::compute(const TimeSeries& cycle, const Datasheet&) const {
    return charge_Ah(cycle, Direction::Charge);
}

double MaxTemperature::compute(const TimeSeries& cycle, const Datasheet&) const {
    if (cycle.empty()) {
        return kNaN;
    }
    return cycle.max(Quantity::Temperature);
}
```

```cpp
CycleAnalyzer CycleAnalyzer::withDefaultMetrics() {
    CycleAnalyzer analyzer;
    analyzer.add(std::make_unique<DischargeCapacity>());
    analyzer.add(std::make_unique<ChargeCapacity>());
    analyzer.add(std::make_unique<DischargeEnergy>());
    analyzer.add(std::make_unique<CoulombicEfficiency>());
    analyzer.add(std::make_unique<StateOfHealth>());
    analyzer.add(std::make_unique<InternalResistance>());
    analyzer.add(std::make_unique<MaxTemperature>());
    return analyzer;
}
```

Résistance interne. Parcourez le cycle et trouvez le premier point k tel que le point précédent soit au repos ($|I_{k-1}| < \mathrm{C}/50$) et le point k en décharge ($I_k < -\mathrm{C}/50$). Alors $R = 1000\,(V_{k-1} - V_k)/(I_{k-1} - I_k)$ en mΩ. Sans un tel passage, renvoyez NaN (`std::numeric_limits<double>::quiet_NaN()`).

Attention

Trois sources d'erreur classiques : les unités (secondes → heures), le signe du courant en décharge, et les seuils « exactement zéro » — un courant mesuré au repos vaut ±0,0002 A, jamais 0.

Validation

`ctest -R etape5` — 13 tests, comparés à la vérité terrain à 0,5 % (5 % pour la résistance). L'onglet *Indicateurs* se remplit.

Étape 6 — État de charge et sécurité des plans de test

Objectif

Écrire un algorithme qui parcourt une série en gardant un état, et traduire des règles de sécurité en code.

Fichiers : `core/src/Estimators.cpp`, `TestPlan.cpp`.

1. SOC par comptage coulombien. `CoulombCountingSoc::estimate(série, ds)` renvoie une valeur de SOC (en %) par point de mesure :

$$\mathrm{SOC}_0 = \mathrm{SOC}_{\text{initial}}, \qquad \mathrm{SOC}_k = \mathrm{SOC}_{k-1} + 100\,\frac{\frac{1}{2}(I_{k-1} + I_k)(t_k - t_{k-1})}{3600\,Q_{\text{nom}}}$$

puis, à chaque point : si $V_k \geq V_{\max} - 0{,}01$ et $0 < I_k \leq 0{,}06\,Q_{\text{nom}}$ (fin de charge CV), recaler à 100 % ; enfin borner à $[0, 100]$ (`std::clamp`). Série vide : vecteur vide.

Indice

Le recalage corrige la dérive : le SOC initial est inconnu (la cellule du premier cycle part de 30 %, pas de 0), mais une fin de charge CV, elle, est reconnaissable. Regardez la courbe SOC dans l'application : le saut au premier cycle, c'est le recalage.

2. Vérifier un plan de test. Un plan qui dépasse les limites de la fiche technique peut détruire la cellule — ou pire. Dans CellSight, l'assistant qui *génère* des plans de test passera par cette vérification avant toute exécution. `TestPlan::checkAgainst(ds)` renvoie un message par violation (vecteur vide = plan conforme) :

Étape	Règles
`ChargeCC`	courant $\leq$ courant de charge max ; tension $\leq V_{\max}$
`ChargeCV`	tension $\leq V_{\max}$
`DischargeCC`	courant $\leq$ courant de décharge max ; tension $\geq V_{\min}$
`Rest`	durée > 0

Validation

`ctest -R etape6` — 8 tests. Le plan `TP-FAST-NMC-3C` des données doit produire exactement 3 violations. L'onglet *Fiche* indique si le plan de chaque cellule est conforme.

Étape 7 — Export CSV

Objectif

Implémenter une interface (`Exporter`) et écrire un fichier texte structuré.

Fichier : `core/src/Exporter.cpp`. L'export MATLAB est fourni (format binaire MAT niveau 4, lisible par `load` dans MATLAB et par `scipy.io.loadmat` en Python). Écrivez `CsvExporter` : `name()` (`"CSV"`), `extension()` (`"csv"`, sans le point) et `write(série, chemin)`, qui produit exactement cet en-tête puis une ligne par mesure, séparateur virgule et point décimal :

```
time_s,current_A,voltage_V,temperature_C,cycle,step
1,1.2494,3.6576,24.96,1,1
```

Lever `std::runtime_error` si le fichier ne peut pas être créé (dossier inexistant, par exemple).

Indice

Par défaut, un flux n'écrit que 6 chiffres significatifs : un temps de 345 612,5 s deviendrait `345612`. Voyez `out.precision(...)`.

Validation

`ctest -R etape7` — 4 tests.

Étape 8 — L'interface graphique

Objectif

Compléter une vraie interface Qt : connecter des signaux à des slots, utiliser les dialogues standards et ouvrir un document avec l'application du système.

Fichier : `gui/MainWindow.cpp`. L'interface est construite ; il reste quatre trous, repérés par `TODO Étape 8`.

1. **Les connexions** (`buildConnections`). Sur le modèle des connexions déjà écrites, reliez : la liste déroulante des indicateurs (`metricCombo_`) à `updateMetricChart` ; l'action « Exporter en CSV » (`exportCsvAction_`) à `onExportCsv` ; le bouton de la fiche (`datasheetButton_`) à `onOpenDatasheet`.
2. **La fiche** (`showCellInfo`). Quatre champs sont remplis ; complétez la chimie, la plage de tension (« 2.50 V – 4.20 V »), le banc (`TestBench::describe`), la voie et l'emplacement.
3. **L'export CSV** (`onExportCsv`). Sur le modèle de `onExportMat` ci-dessous, mais en n'exportant *que les cycles affichés* dans l'onglet *Mesures* (`firstCycle_` à `lastCycle_`). Proposez un nom de fichier qui les rappelle.

4. **La fiche technique** (`onOpenDatasheet`). Si le PDF existe (`QFileInfo::exists`), l'ouvrir avec `QDesktopServices::openUrl(QUrl::fromLocalFile(...))` ; sinon, afficher un `QMessageBox::warning`.

```cpp
void MainWindow::onExportMat() {
    if (!current_ || !current_->hasData()) {
        QMessageBox::information(this, "Export", "Aucune mesure chargée.");
        return;
    }
    const QString path = QFileDialog::getSaveFileName(
        this, "Exporter en MATLAB", qs(current_->id()) + ".mat", "Fichiers MATLAB (*.mat)");
    if (path.isEmpty()) {
        return;  // l'utilisateur a annulé
    }
    try {
        cs::MatExporter().write(current_->data(), nativePath(path));
        statusBar()->showMessage("Mesures exportées dans " + path);
    } catch (const std::exception& e) {
        QMessageBox::warning(this, "Export", "Échec de l'export :\n" + qs(e.what()));
    }
}
```

Indice

Dans `connect`, un signal qui transmet un argument (l'indice choisi) peut être relié à un slot qui n'en prend pas : Qt ignore l'argument en trop. `QString::fromStdString` et la fonction `qs(...)` du fichier convertissent une `std::string` en `QString`. Pour passer un chemin à un `std::ofstream`, utilisez `nativePath(...)` (fournie) : sous Windows, un chemin accentué ne s'écrit pas en UTF-8.

Validation

Pas de test automatique : vérifiez à la main que chaque indicateur s'affiche, que l'export CSV produit un fichier relisible (ouvrez-le dans un tableur), que la fiche PDF s'ouvre, et que l'utilisateur « invite » ne peut rien exporter. Testez aussi l'annulation des dialogues.

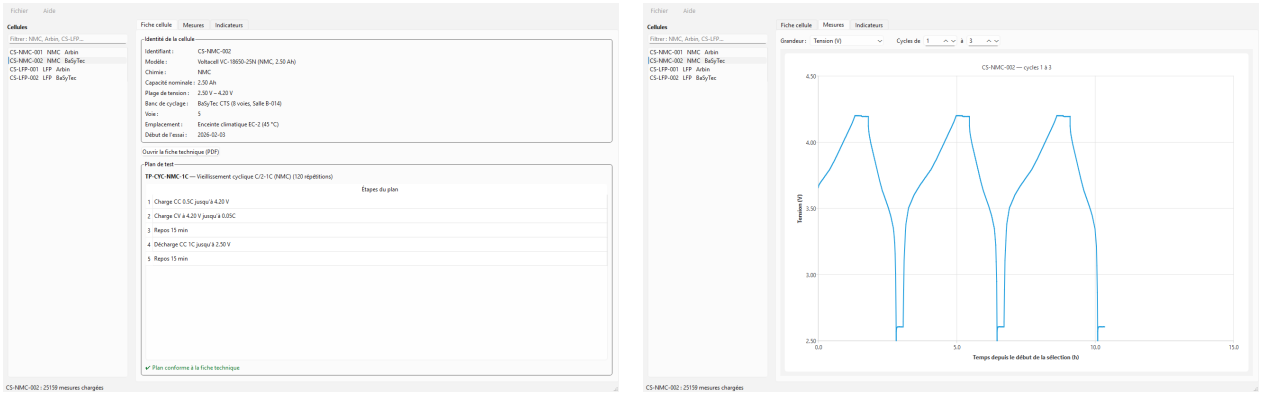


FIGURE 6 – Les onglets *Fiche cellule* et *Mesures* de l'application terminée.

Étape 9 — Bonus

Au choix, dans l'ordre que vous voulez. Décrivez chaque bonus dans le rapport.

1. **Durée de vie restante** (`LinearRulEstimator::estimate`, `ctest -R etape9`). Ajuster une droite SOH $= a\,n + b$ par moindres carrés sur les `window` derniers cycles valides (NaN ignorés) :

$$a = \frac{N\sum n\,s - \sum n \sum s}{N\sum n^2 - (\sum n)^2}, \qquad b = \frac{\sum s - a\sum n}{N}, \qquad n_{\text{fin}} = \frac{80 - b}{a}.$$

   Résultat invalide si moins de 3 points ou si $a \geq 0$. L'onglet *Indicateurs* affiche alors la durée de vie restante.
2. **Exporter le graphique en PNG** (`onExportChartPng`) : `QWidget::grab()` renvoie une image du widget.
3. **Un nouvel indicateur**, par exemple le rendement énergétique ou la durée de charge : une classe dérivée de `Metric`, une ligne dans `withDefaultMetrics()`… et constatez qu'il apparaît dans l'interface sans autre modification. (Seul cas où vous pouvez modifier un en-tête.)
4. **Chargement en arrière-plan** (cours 5). Le chargement d'une cellule fige l'interface une fraction de seconde. Lancez `loadData` dans un `QtConcurrent::run` et récupérez le résultat avec un `QFutureWatcher`. Attention : on ne touche aux widgets que depuis le thread principal.

7. Et après ? Votre code dans CellSight

CellSight Lite lit des fichiers et calcule des indicateurs simples. La plateforme complète du laboratoire réutilise exactement le même découpage :

Dans CellSight Lite	Dans la plateforme CellSight
`CsvRepository`	un dépôt qui interroge la base de données centrale par une API web
`DataImporter`	le service d'ingestion qui surveille les dossiers des cycleurs
`Metric`, `SocEstimator`	en plus : des modèles d'IA qui estiment SOC, SOH et RUL, et détectent les anomalies
`Datasheet`, `Cell`, `TestPlan`	les concepts de l'ontologie du domaine (BattINFO), partagés avec d'autres laboratoires
`TestPlan::checkAgainst`	le garde-fou de l'assistant conversationnel qui propose des plans de test

Les interfaces que vous avez implémentées sont des points d'extension : un estimateur d'IA n'est qu'une classe dérivée de plus de `SocEstimator`.

A. Problèmes fréquents

CMake : « Qt 6 introuvable ». Qt Charts n'a pas été installé (relancer l'outil de maintenance de Qt et cocher *Qt Charts*), ou le kit choisi dans Qt Creator n'est pas un kit Qt 6.

Tous les tests d'une étape sont « A FAIRE ». Il reste un `throw NotImplemented` : effacez-le avec le TODO.

Un test de l'étape 5 échoue d'un facteur 3600. Les temps BaSyTec ne sont pas convertis en secondes (étape 3), ou la division par 3600 manque.

« Impossible d'ouvrir … ». Le dossier `data/` a été déplacé : il doit rester à côté de `CMakeLists.txt`. L'application accepte aussi un dossier de données en argument.

Sous Linux, les nombres sont mal lus. La locale française écrit `3,6` : `gui/main.cpp` rétablit la locale « C » pour les nombres. Ne retirez pas cette ligne.